Video RAG Q&A; Bot — Solution Writeup

Project Code Repository: https://github.com/molubhai08/Video_Rag
Demo Video Link: Google Drive Video
Deployment Note: Not currently live on cloud due to YouTube IP blocks on cloud instances (details below). Runs locally.

1. The Workflow, Prompts, and Steps

The system implements a **Two-Level Retrieval-Augmented Generation (RAG)** pipeline to pinpoint answers and timestamps from long video transcripts without requiring database-backed vector store infrastructure.

- **1. Extraction:** The user inputs a YouTube URL. The system extracts the 11-character video ID.
- **2. Transcript Processing & Local Caching:** Checks local JSON cache first. If it's a miss, fetches the transcript via the youtube-transcript-api client, groups elements into semantic chunks of ~450 words (with 100 words overlap), and writes to local cache.
- **3. Level 1 Retrieval:** Fits a lightweight TfidfVectorizer (unigrams + bigrams) on the fly over chunks, computes cosine similarity against the query vector, and retrieves the top 3 chunks.
- **4. Level 2 Refinement:** Scores individual snippets inside the best-matching chunk using keyword word-overlaps to pick the exact starting seconds timestamp.
- **5. Answer Generation:** Sends the user query + top 3 context chunks to the Groq API (using the llama-3.1-8b-instant model).
- **6. Seeking Interface:** Displays the response in the UI. When the user clicks the timestamp link, the app triggers HTML5 postMessage script calls to automatically seek the YouTube iframe player.

System Prompt Context for Groq LLM

```
System Prompt:
You are an expert podcast analyst. Answer the user's question based ONLY on the provided
transcript segments. Be concise and insightful (2-4 sentences). If the answer isn't in the
transcript, say so honestly. Do NOT make up information.
```

2. Tools & Models Used

- **Frontend:** Vanilla HTML5, CSS3, & Modern Javascript (Inter & Space Grotesk typography, glassmorphism, custom interactive controls).
- **Backend Framework:** Flask (Python 3.10+) for APIs, thread execution, and caching.

- **Transcript Parsing:** youtube-transcript-api configured with custom cookie headers to bypass basic scraping filters.
- **Retrieval Engine:** Scikit-learn (TF-IDF vectorizer + Cosine Similarity computations).
- **LLM Engine:** Groq API with llama-3.1-8b-instant for sub-300ms response latencies.

3. Accuracy Check & Verification

- **Grounding Audit:** Checked responses to ensure no hallucinations occur. The Groq model returns negative responses correctly when the transcript does not contain the answer.
- **Timestamp Pinpointing:** Validated against Lex Fridman and Elon Musk interviews. Verified generated timestamps match ground-truth timestamps within 5 seconds.
- **Cache Testing:** Confirmed that subsequent questions on cached video IDs load immediately (< 10ms) from local memory rather than hitting YouTube rates.

4. Current Limitations & Future Improvements

A. TF-IDF for Semantic Search

Limitation: We utilized TF-IDF because it is lightweight, requires no heavy external machine learning models/downloads, and runs in milliseconds on standard CPU units. However, it relies entirely on keyword matching. If the user asks a question using synonyms instead of exact words used in the video, TF-IDF might fail to rank the correct chunk highest.
Improvement: In future versions, we will transition to dense semantic vector embeddings (e.g. OpenAI's text-embedding-3-small or Hugging Face's BGE-m3) to match conceptually.

B. Local JSON Caching

Limitation: Transcripts are cached as local .json files, and active tasks are tracked using in-memory Python dictionaries with thread locks. This does not scale horizontally across multi-worker cloud servers because worker processes do not share memory and local disk storage is ephemeral.
Improvement: We will integrate **Redis** for task status and transcript caching to support highly available, horizontally scaled cloud configurations.

C. Cloud Rate Limits (YouTube Blocks)

Limitation: Deployment to cloud services (like Azure App Service or AWS) throws errors because YouTube heavily rate-limits/blocks datacenter IP blocks.
Improvement: Implement rotating residential proxy configuration options (e.g. Webshare integration) inside the YouTube client HTTP agent.

D. AI Multimodal Blindness (Text-Only Limitation)

Limitation: The AI model struggles with visual reference questions because it performs search on text transcripts only. If a host points to a slide or prototype and says, *'As you can see here, this is the main issue'*, the text-only RAG misses the visual context of what *'this'* refers to.
Improvement: In future versions, we can sample video frames (e.g., every 5 seconds) and pass them to a Vision-Language Model (VLM like Gemini Flash) to append visual summaries to the text transcripts.